CATLAB

DOCUMENTO DE ANÁLISE

1. Resumo **4**
2. Objetivo **5**
2.1 Objetivo Geral 5
2.2 Objetivos Específicos 5
3. Escopo **6**
3.1 Incluído 6
3.2 Não Incluído 6
3.3 Premissas 6
3.4 Restrições 6
4. API Utilizada **7**
4.1 Endpoints 7
5. Requisitos **8**
5.1 Requisitos Funcionais 8
5.1.1 - RF01: Exibição de Descrição Inicial 8
5.1.2 - RF02: Formulário de Geração de Conteúdo 8
5.1.3 - RF03: Valor Padrão do Seletor (select) 8
5.1.4 - RF04: Clique do Botão de “Gerar” 8
5.1.5 - RF05: Comportamento da Opção “Aleatório” 8
5.1.7 - RF06: Indicador de Carregamento da Mídia 9
5.1.8 - RF08: Limitação de Caracteres 9
5.1.9 - RF09: Controle de Nova Requisição 9
5.2 Requisitos Não Funcionais 9
5.2.1 Segurança 9
5.2.1.1 - RNF01: Sanitização de Entrada 9
5.2.1.2 - RNF02: Não exposição de erros internos 9
5.2.1.3 - RNF03: Controle de requisições 9
5.2.2 Performace 10
5.2.2.1 - RNF04: Indicador de Processamento 10
5.2.3 Usabilidade 10
5.2.3.1 - RNF05: Interface Responsiva 10
5.2.4 Arquitetura 10
5.2.4.1 - RNF06: Separação de responsabilidades 10
5.2.4.2 - RNF07: Modularidade 10
6. Arquitetura **10**
6.1 Componentes 11
6.1.1 Interface Web (front-end) 11
6.1.2 API Externa 11
6.2 Fluxo de Funcionamento 11
6.3 Características da Arquitetura Atual 11
7. Tecnologias Utilizadas **12**
8. Modelagem de Banco de Dados **12**
8.1 Visão Geral 12
8.2 Entidades 12

8.2.1 Usuário..... 12
8.2.2 Requisições..... 13
8.3 Relacionamento..... 13
9.3.1 Usuário x Requisição..... 13
8.4 Considerações de Segurança..... 13
9. Decisões Técnicas..... 14
9.1 Angular (framework front-end)..... 14
9.2 Typescript..... 14
9.3 Tailwind..... 14
9.4 Integração direta com API externa..... 14
9.5 Controle de Requisições..... 14
9.1 React (framework front-end)..... 15
10. Evoluções Planejadas..... 15
10.1 Desenvolvimento de API Própria..... 15
10.2 Sistema de Autenticação de Usuários..... 15
10.3 Persistência de Histórico de Requisições..... 15
10.4 Área Administrativa..... 15
11. Conclusão..... 16

1. Resumo

Este projeto consiste no desenvolvimento de uma aplicação web interativa voltada para fins demonstrativos e educacionais, com o objetivo de apresentar boas práticas de desenvolvimento front-end e consumo de API pública.

A aplicação tem como proposta principal consumir a CATAAS para exibir imagens de gatos, oferecendo ao usuário uma experiência leve, divertida e responsiva. O sistema permitirá a geração dinâmica de conteúdo, como imagens aleatórias, incluindo textos, GIFs e pequenas frases personalizadas.

O projeto foi desenvolvido com foco em organização de código, escalabilidade e experiência do usuário, servindo como um portfólio técnico para demonstrar conhecimentos em arquitetura de software, integração com APIs REST, componentização, gerenciamento de estado e responsividade.

Além disso, o projeto não possui fins comerciais e tem como principal finalidade a apresentação de habilidades técnicas, servindo como base para futuras evoluções, como inclusão de novas APIs, persistência de dados e melhorias na interface.

2. Objetivo

2.1 Objetivo Geral

Desenvolver uma aplicação web moderna e responsiva com o objetivo de demonstrar habilidades técnicas em desenvolvimento de software, por meio da integração com APIs públicas e da implementação de boas práticas de arquitetura, organização de código e experiência do usuário.

2.2 Objetivos Específicos

- Consumir APIs CATAAS (Cat as Service) para obtenção dinâmica de imagens.
- Implementar uma interface intuitiva, responsiva e de fácil utilização.
- Demonstrar o uso de componentização, serviços e separação de responsabilidades no front.
- Aplicar boas práticas de versionamento de código e estruturação de projeto.
- Garantir boa performance no carregamento e atualização do conteúdo.
- Servir como projeto de portfólio técnico para apresentação de competências em front-end.
- Possibilitar futuras evoluções do sistema, como persistência de dados, novas integrações e melhorias visuais.

3. Escopo

3.1 Incluído

O projeto contempla o desenvolvimento de uma aplicação web com as seguintes funcionalidades e características:

- Consumo exclusivo da API CATAAS.
- Geração de imagens e GIFs aleatórios de gatos.
- Customização de imagens com texto.
- Interface responsiva.
- Modo claro/escuro.

3.2 Não Incluído

As seguintes funcionalidades **não fazem parte do escopo desta versão** do projeto e poderão ser implementadas futuramente:

- Sistema de autenticação e cadastro de usuários;
- Persistência de dados em banco de dados remoto;
- Painel administrativo;
- Funcionalidades offline avançadas;
- Customização avançada de temas e cores;
- Testes automatizados avançados (unitários e e2e).

3.3 Premissas

- O projeto utilizará exclusivamente API pública e gratuita.
- A aplicação será desenvolvida com foco em fins educacionais e de portfólio.
- O ambiente de execução principal será o navegador web.
- O versionamento do código será feito utilizando Git e repositório público.

3.4 Restrições

- Dependência da disponibilidade e estabilidade das APIs públicas utilizadas.
- Limitações de desempenho impostas por serviços externos.
- Prazo e escopo ajustados para desenvolvimento individual.

4. API Utilizada

Para tornar o processo de desenvolvimento mais “agradável” e ser também uma forma na qual consiga me divertir e ao mesmo tempo demonstrar tudo o que aprendi nesses anos desenvolvendo, escolhi algo que mudou um pouco a minha vida, minha forma de enxergar tecnologia e criatividade aplicada ao desenvolvimento.. escolhi uma api chamada CAATAS (Cat as Service).

Ela consiste em gerar imagens e gifs randômicos de gatinhos e até por textos dentre o conteúdo gerado. Também é possível realizar filtros, tamanhos e larguras diferentes de acordo com as especificações do usuário. Neste caso, iremos trabalhar somente com as funcionalidades de:

- Gerar imagens com/sem texto
- Gerar gifs com/sem texto.

4.1 Endpoints

O link base da api é este: Cat as a service (CATAAS).

Para este projeto, inicialmente serão utilizados os seguintes endpoints:

Endpoints	**Descrição**
/cat	Traz um gatinho aleatório.
/cat/gif	Gera um gif de um gatinho aleatório.
/cat/says/:text	Gera uma imagem de um gatinho, onde o :text é o texto ser mostrado na imagem.
/cat/gif/says/:text	Gera uma gif de um gatinho, onde o :text é o texto a ser ostrado.

Tabela 1: Lista de endpoints com suas respectivas descrições.

5. Requisitos

Nesta seção, será dividida entre **Requisitos Funcionais** e **Requisitos Não Funcionais.**

O **Requisito Funcional** (que vamos chamar de **RF**), consiste em abranger todas as necessidades a serem atendidas pelo CatLab, ou seja, o que ele faz, qual função será atendida, já o **Requisito Não Funcional** (que por sua vez, chamaremos de: **RNF**), é um descritivo do comportamento do sistema.

5.1 Requisitos Funcionais

5.1.1 - RF01: Exibição de Descrição Inicial

O sistema deve exibir, na página inicial, uma breve descrição explicando o funcionamento da aplicação.

5.1.2 - RF02: Formulário de Geração de Conteúdo

Na tela inicial do site, após a explicação, deverá ter um formulário contendo:

- Campo de texto para a inserção de mensagem personalizada;
- Campo de Tipo Imagem, uma seleção, com as opções: Aleatório, Imagem ou Gif;
- Botão de Gerar.

5.1.3 - RF03: Valor Padrão do Seletor (select)

A opção “Aleatório” deve ser selecionada por padrão ao acessar o site.

5.1.4 - RF04: Clique do Botão de “Gerar”

Ao acionar o botão “Gerar”, o sistema deve:

- Processar os valores informados no formulário;
- Realizar a requisição à API externa CATAAS;
- Exibir a mídia retornada na própria página, abaixo do formulário.

5.1.5 - RF05: Comportamento da Opção “Aleatório”

Quando a opção **“Aleatório”** estiver selecionada, o sistema deve definir automaticamente, de forma randômica, se será gerada uma imagem estática ou um GIF.

5.1.7 - RF06: Indicador de Carregamento da Mídia

Depois de pressionar o botão “Gerar”, o sistema deverá exibir a área da imagem no qual está sendo gerada, com um indicador de “carregando”.

5.1.8 - RF08: Limitação de Caracteres

O sistema deve limitar o campo de mensagem a no máximo **50 (cinquenta) caracteres**.

5.1.9 - RF09: Controle de Nova Requisição

Após o acionamento do botão “Gerar”, o sistema deve:

- Desabilitar o botão por 30 segundos;
- Alterar seu conteúdo visual para um indicador de carregamento;
- Exibir um temporizador em contagem regressiva informando o tempo restante para nova requisição.

5.2 Requisitos Não Funcionais

5.2.1 Segurança

5.2.1.1 - RNF01: Sanitização de Entrada

O sistema deve validar e sanitizar o texto informado pelo usuário antes de enviá-lo à API externa CATAAS.

5.2.1.2 - RNF02: Não exposição de erros internos

A aplicação não deve exibir mensagens técnicas detalhadas ou informações internas da mesma em caso de falha.

5.2.1.3 - RNF03: Controle de requisições

O sistema deve impedir múltiplas requisições consecutivas em curto intervalo de tempo por meio de mecanismo de bloqueio temporário do botão de geração e validação da api.

5.2.2 Performace

5.2.2.1 - RNF04: Indicador de Processamento

Ao clicar no botão de “Gerar”, o sistema deverá exibir um gif no estilo de “carregando” com as seguintes características:

- O CatLab deve apresentar retorno visual imediato do gif.
- A plataforma deve fornecer indicação visual clara durante o processamento da requisição.

5.2.3 Usabilidade

5.2.3.1 - RNF05: Interface Responsiva

A interface deve ser adaptável a diferentes tamanhos de tela (desktop e dispositivos móveis).

5.2.4 Arquitetura

5.2.4.1 - RNF06: Separação de responsabilidades

A aplicação deve manter a separação entre camada de interface, lógica de processamento e integração com API externa.

5.2.4.2 - RNF07: Modularidade

A estrutura do código deve permitir manutenção e expansão futura sem necessidade de reestruturação completa.

6. Arquitetura

A aplicação **CatLab** é uma solução web executada inteiramente no lado do cliente (client-side), não possuindo backend próprio na versão atual, de forma que, o processamento das requisições é realizado diretamente no navegador do usuário, que se comunica com a API externa CATAAS para geração das imagens.

Por virtude disto, a arquitetura adotada é do tipo: **Cliente -> Api Externa**, onde não será necessário a persistência de dados, autenticação de usuários ou armazenamento de informações na versão atual do sistema.

6.1 Componentes

6.1.1 Interface Web (front-end)

Responsável por:

- Exibir a interface ao usuário;
- Coletar os dados inseridos no formulário;
- Controlar estados visuais (carregamento, temporizador, exibição de mídia);
- Realizar requisições HTTP para a API externa;
- Exibir a resposta retornada.

6.1.2 API Externa

Serviço responsável pela geração das imagens com texto personalizado, fornecido pela API CATAAS.

A comunicação ocorre via requisição HTTP, retornando a mídia solicitada conforme os parâmetros enviados.

6.2 Fluxo de Funcionamento

A seguir, encontra-se o fluxo de funcionamento da aplicação:

1. Usuário entra no site;
2. Caso optar por preencher algo do formulário:
   a. Mensagem
      i. Se preencher a mensagem, valida a quantidade mínima e máxima do mesmo.
      ii. Se não preencher, gera sem mensagem.
   b. Tipo de Mídia
      i. Se selecionar a opção, gera a mídia conforme o que foi selecionado
      ii. Se não, gera de forma aleatória
3. Requisição é enviada;
4. Loading é exibido;
5. Resposta é tratada;
6. Mídia renderizada;
7. Botão fica desabilitado e entra em cooldown de 30s.

6.3 Características da Arquitetura Atual

A arquitetura atual apresenta as seguintes características:

- Ausência de backend próprio;
- Ausência de banco de dados;
- Processamento integral no navegador;
- Dependência de serviço externo para geração de conteúdo;
- Estrutura modular preparada para futura expansão.

7. Tecnologias Utilizadas

Abaixo, estão todas as tecnologias utilizadas no desenvolvimento da aplicação **CatLab**:

- **HTML5** – Estrutura semântica da aplicação;
- **TypeScript** – Tipagem estática e maior segurança no desenvolvimento;
- **TailwindCSS** – Estilização utilitária e responsiva;
- **Angular** – Implementação baseada em componentes para a primeira versão;
- **React** – Implementação alternativa para estudo;
- Integração com a API externa CATAAS.

8. Modelagem de Banco de Dados

8.1 Visão Geral

O site **CatLab** inicialmente não irá conter banco de dados, entretanto, por questão de estudo e demonstrar conhecimento técnico nesta parte, foi realizado um modelamento para futura implementação.

8.2 Entidades

8.2.1 Usuário

Nome da Tabela: usuário

Campos:

- id
- apelido
- senha
- create_at

- update_at

Observações:

- ID PK;
- Apelido, string único
- senha: hash, criptografado
- update/create_at automáticos

8.2.2 Requisições

Nome da Tabela: requisicao

Campos:

- id
- user_id
- mensagem
- tipo_midia
- url
- data_hora

Observações:

- ID PK;
- user_id, fk de usuário
- url: É a url da requisição, caso precise pegar o conteúdo dela de novo.
- data_hora automático

8.3 Relacionamentos

9.3.1 Usuário x Requisição

Um usuário pode conter várias requisições que por sua vez, cada requisição possui apenas um usuário, sendo assim com o relacionamento de **1:N**.

8.4 Considerações de Segurança

A seguir, algumas considerações relacionada a segurança aos dados do banco:

- A senha do usuário deverá ser criptografada (hash bycript, etc)
- O campo de Apelido em usuário, deverá ser único;
- Deverá ser feita a validação server-side do usuário

- Controle de acesso para a área administrativa.

9. Decisões Técnicas

9.1 Angular (framework front-end)

Escolhido devido minha familiaridade prévia com o framework. Além disso, possui forte integração com o typescript, ademais por seu escalonamento e facilidade de componentização.

9.2 Typescript

A aplicação será desenvolvida usando o Typscript para garantir maior segurança se tratando do retorno e envio de dados a fim de reduzir erros comuns de tipagem presentes no Javascript puro.

9.3 Tailwind

Aceleração do design responsivo com uso de classes diretamente no html que agiliza o processo e também por uma questão de me desafiar.

9.4 Integração direta com API externa

Nesta primeira versão, a aplicação realiza requisições diretamente para a API CATAAS, sem a utilização de backend intermediário.

Essa decisão foi tomada para simplificar a arquitetura inicial e permitir foco na implementação da interface e na lógica de interação com o usuário.

9.5 Controle de Requisições

Foi implementado um intervalo mínimo entre requisições para evitar múltiplas chamadas consecutivas à API externa, prevenindo abuso do serviço e melhorando a experiência do usuário.

9.1 React (framework front-end)

Esta escolha foi por um desafio técnico, no qual gostaria de testar e explorar um novo desafio, como não tenho tanta experiência com este framework, gostaria de explorá-lo.

10. Evoluções Planejadas

10.1 Desenvolvimento de API Própria

Em versões futuras, será desenvolvida uma API intermediária responsável por realizar a comunicação com a API externa CATAAS.

Essa abordagem permitirá maior controle sobre as requisições, além de possibilitar implementação de cache, segurança e monitoramento.

10.2 Sistema de Autenticação de Usuários

Implementação de autenticação baseada em apelido e senha, permitindo que usuários possuam uma identidade dentro da aplicação.

As credenciais devem ser armazenadas utilizando hash seguro.

10.3 Persistência de Histórico de Requisições

Armazenamento das requisições realizadas pelos usuários, permitindo a criação de um histórico de imagens geradas.

10.4 Área Administrativa

Criação de uma interface administrativa onde cada usuário cadastrado poderá ver as suas requisições, alterar seus dados e o administrador conseguir ver os usuários e a quantidade de requisições dos mesmos.

11. Conclusão

O projeto **CatLab** foi desenvolvido com o objetivo de criar uma aplicação web simples capaz de gerar imagens de gatos personalizadas a partir de uma mensagem inserida pelo usuário, utilizando integração com a API pública CATAAS.

Durante o desenvolvimento, foram aplicados conceitos importantes de arquitetura de aplicações web, organização modular de código, controle de requisições e integração com serviços externos. A aplicação foi estruturada utilizando Angular, juntamente com TypeScript e Tailwind CSS, visando manter o código organizado, escalável e de fácil manutenção.

Além da implementação atual, também foram planejadas evoluções futuras para o projeto, incluindo desenvolvimento de uma API própria, persistência de dados, autenticação de usuários e implementação de uma segunda versão utilizando React, permitindo estudo comparativo entre diferentes frameworks.

Dessa forma, o projeto serve tanto como demonstração prática de conhecimentos em desenvolvimento web quanto como base para futuras evoluções em um cenário full stack.